



中山大学 软件工程学院

SUN YAT-SEN UNIVERSITY SCHOOL OF SOFTWARE ENGINEERING

SSE206 《计算机网络》

第三章 传输层

南雨宏

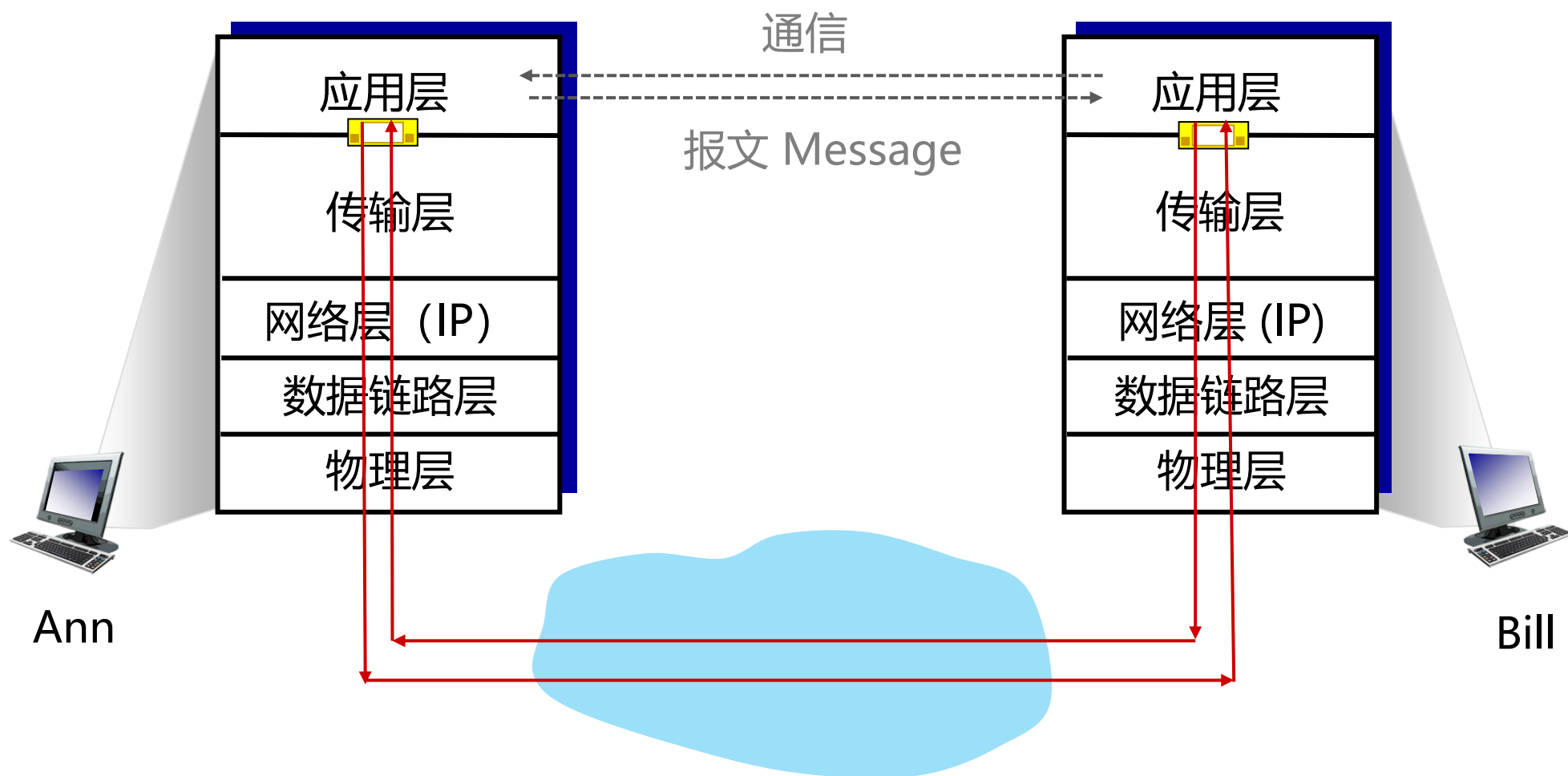
nanyh@mail.sysu.edu.cn

2026-03-31

提纲

- 第 1 章 计算机网络和因特网
- 第 2 章 应用层
- 第 3 章 传输层
- 第 4 章 网络层：数据平面
- 第 5 章 网络层：控制平面
- 第 6 章 链路层与局域网
- 第 7 章 无线网络和移动网络
- 第 8 章 计算机网络中的安全

知识回顾：5 层 Internet 协议栈



第 3 章：传输层 Transport layer

目标:

- 理解传输层工作原理
 - 多路复用 / 解复用
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- 学习 Internet 传输层协议
 - UDP: 无连接的不可靠传输
 - TCP: 面向连接的可靠传输
 - TCP 拥塞控制

第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

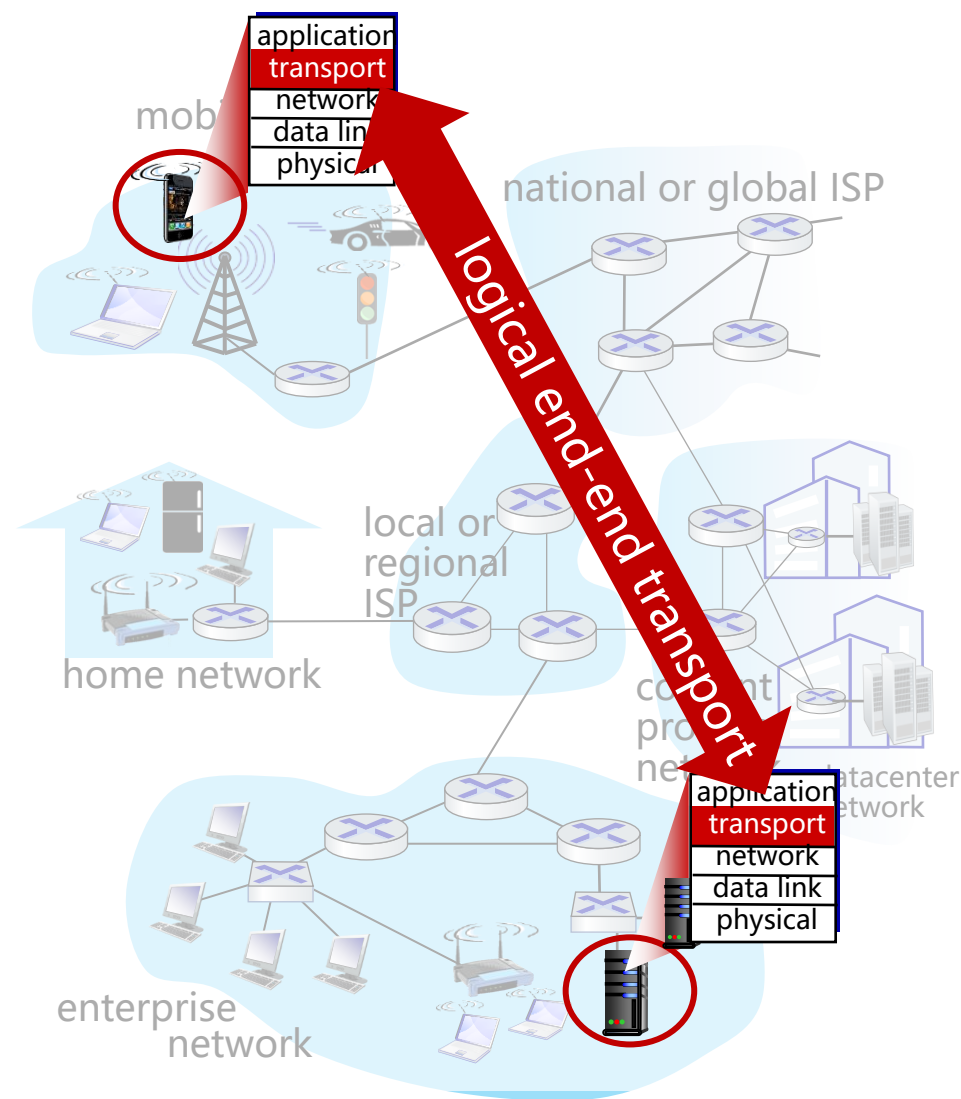
3.6 拥塞控制原理

3.7 TCP拥塞控制

3.8 传输层功能的演化

传输服务和协议

- 为运行在不同主机上的应用进程 (process) 提供**逻辑通信** (logical communication)
- 传输层协议**运行在端系统**
 - 发送方：将应用层的报文 (message) 分成**报文段** (segment)，然后传递给网络层
 - 接收方：将报文段重组为报文，然后传递给应用层
- 有多个 (两个) 传输层协议可供应用选择
 - Internet 协议栈：TCP和UDP



传输层和网络层的关系

东西海岸2个家庭的通信

Ann家的12个孩子给Bill家的12个孩子发信:

- 应用层报文 = 信封中的信件
- 进程 = 孩子
- 主机（端系统）= 家庭
- 传输层协议 = Ann 和 Bill
 - 为家庭中的孩子提供复用、解复用服务
- 网络层协议 = 邮政服务（找到Ann和Bill家庭地址）

传输层和网络层的关系

- 网络层服务: 主机之间的逻辑通信
- 传输层服务: 进程之间的逻辑通信
 - 依赖于网络层的服务: 延时, 带宽
 - 并对网络层的服务进行增强: 数据丢失、顺序混乱

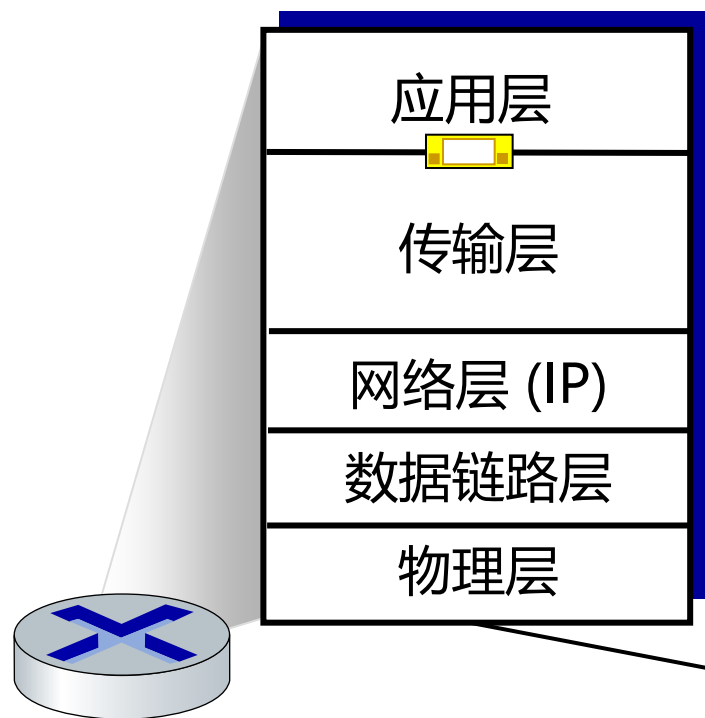
有些服务可以被增强: 不可靠 -> 可靠

有些服务不可以被增强: 带宽、延迟

传输层工作

发送方：

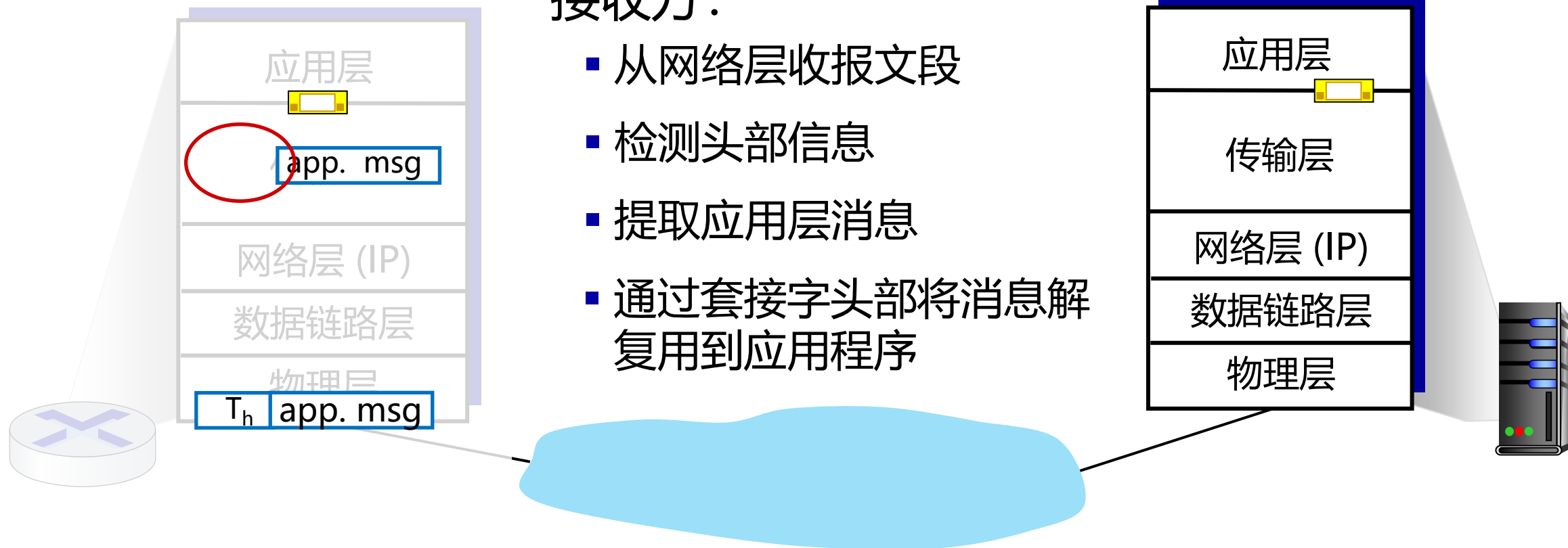
- 传递一个应用层消息
- 确定报文段头部信息
- 生成报文段
- 将报文段传送给网络



传输层工作

接收方：

- 从网络层收报文段
- 检测头部信息
- 提取应用层消息
- 通过套接字头部将消息解复用到应用程序

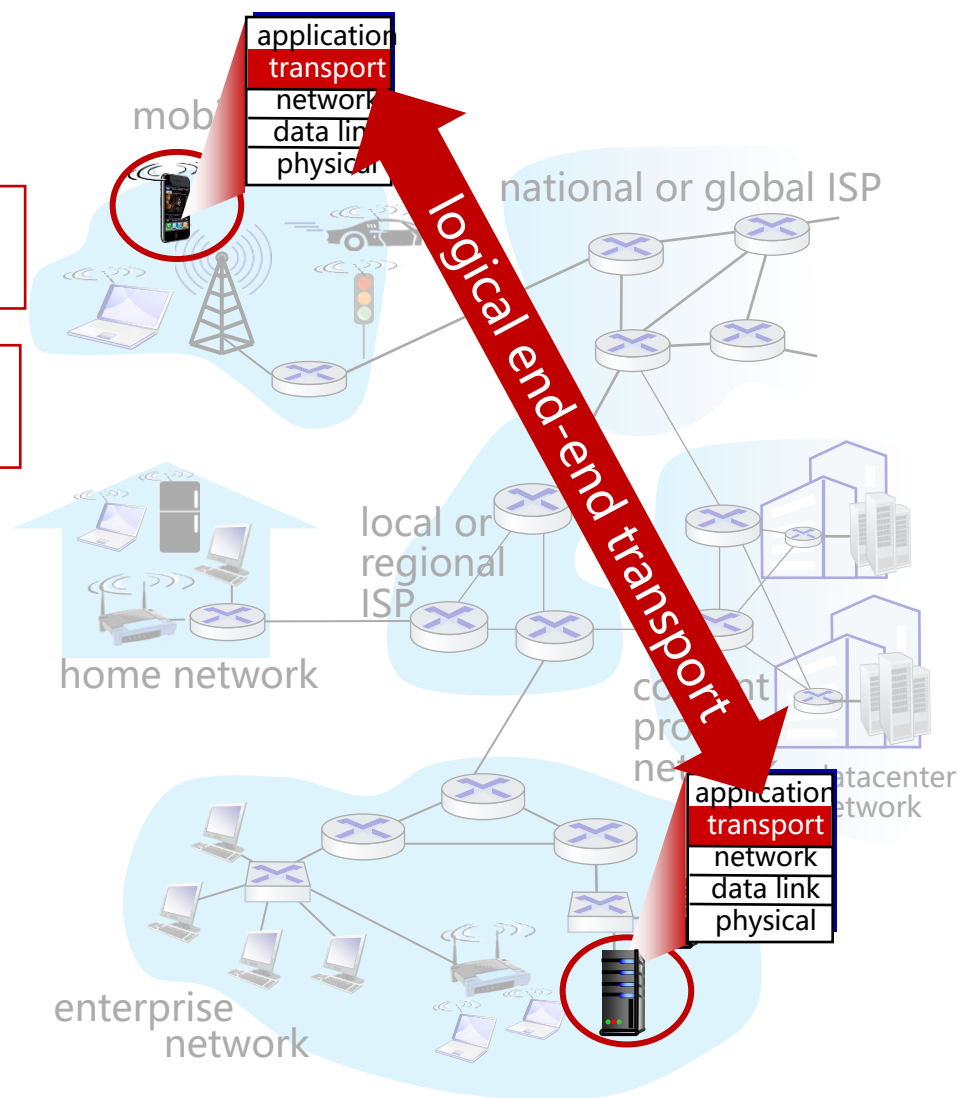


Internet 传输层协议

- 可靠的、保序的传输：TCP
 - 多路复用、解复用
 - 拥塞控制、流量控制
 - 面向连接
- 不可靠、不保序的服务：UDP
 - 多路复用、解复用
 - 没有为尽力而为的IP服务添加更多的其他额外服务
- 都不提供的服务：
 - 延时保证、带宽保证

Transmission Control Protocol
传输控制协议

User Datagram Protocol
用户数据报协议



第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用(多路分解)

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

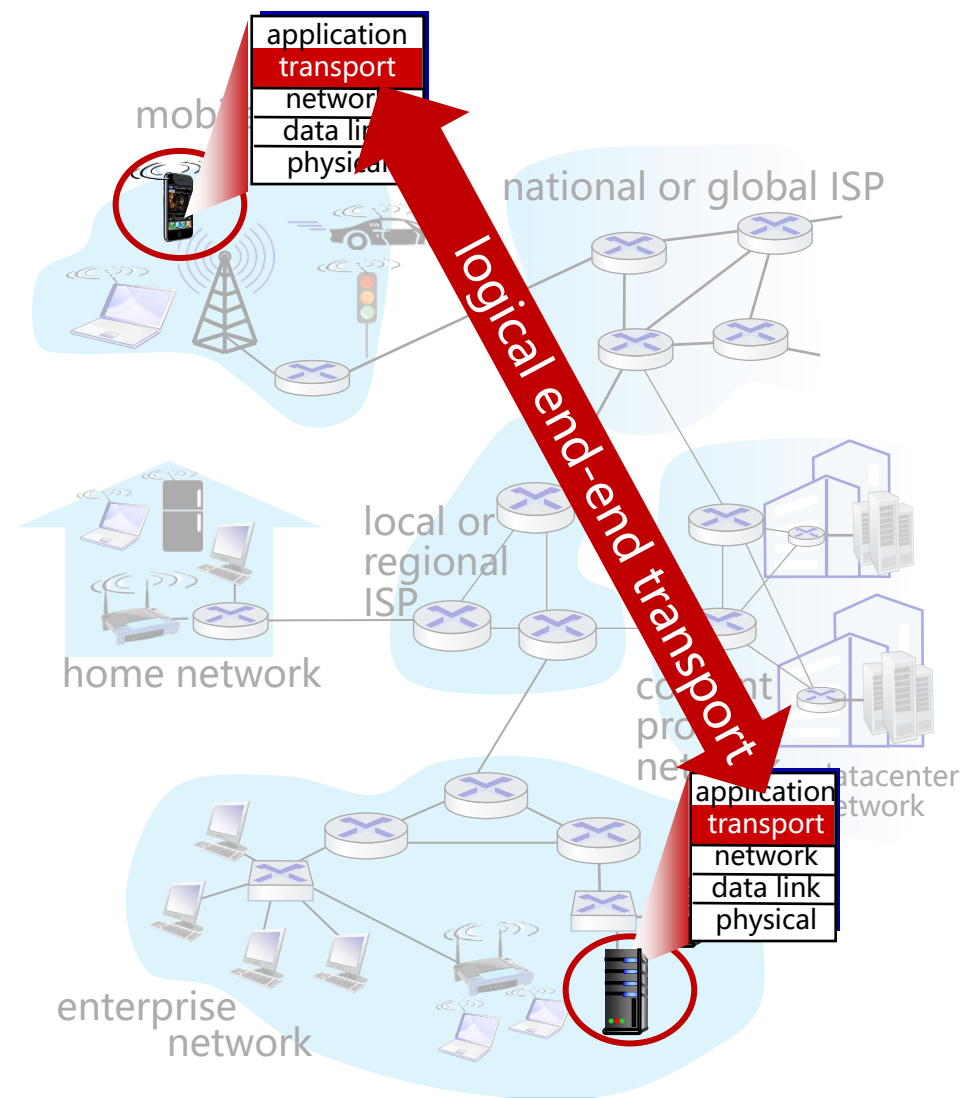
3.6 拥塞控制原理

3.7 TCP拥塞控制

3.8 传输层功能的演化

Internet 传输层协议

- 可靠的、保序的传输：TCP
 - 多路复用、解复用
 - 拥塞控制、流量控制
 - 面向连接
- 不可靠、不保序的服务：UDP
 - 多路复用、解复用
 - 没有为尽力而为的IP服务添加更多的其他额外服务
- 都不提供的服务：
 - 延时保证、带宽保证



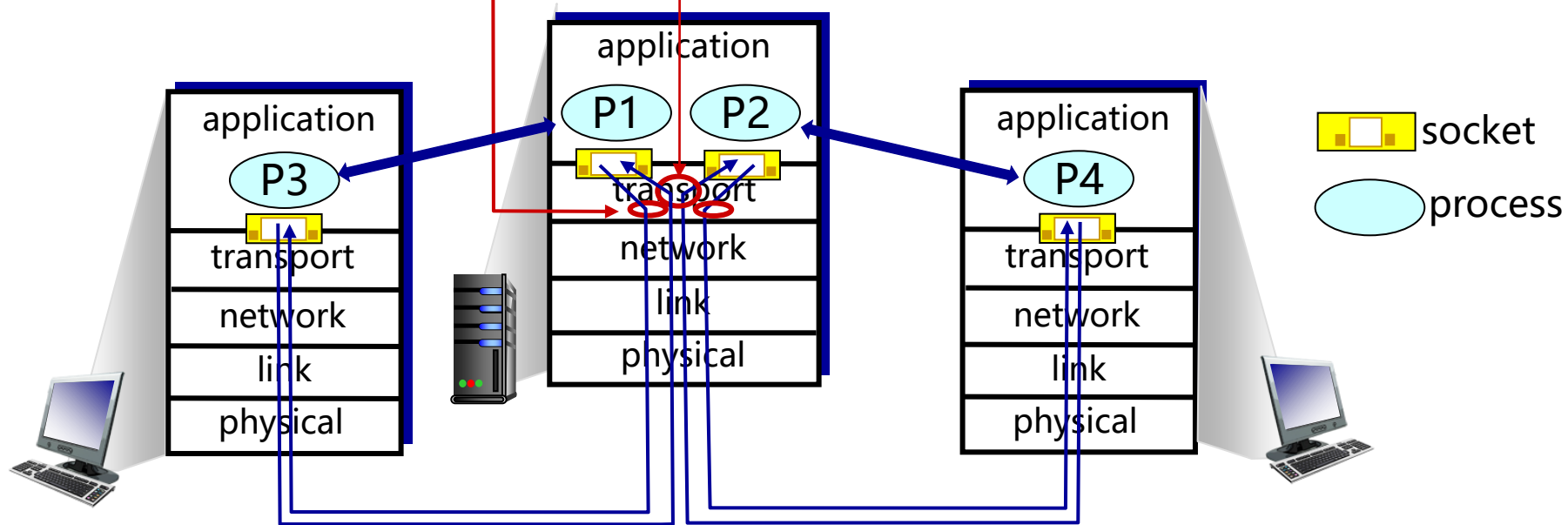
多路复用/解复用

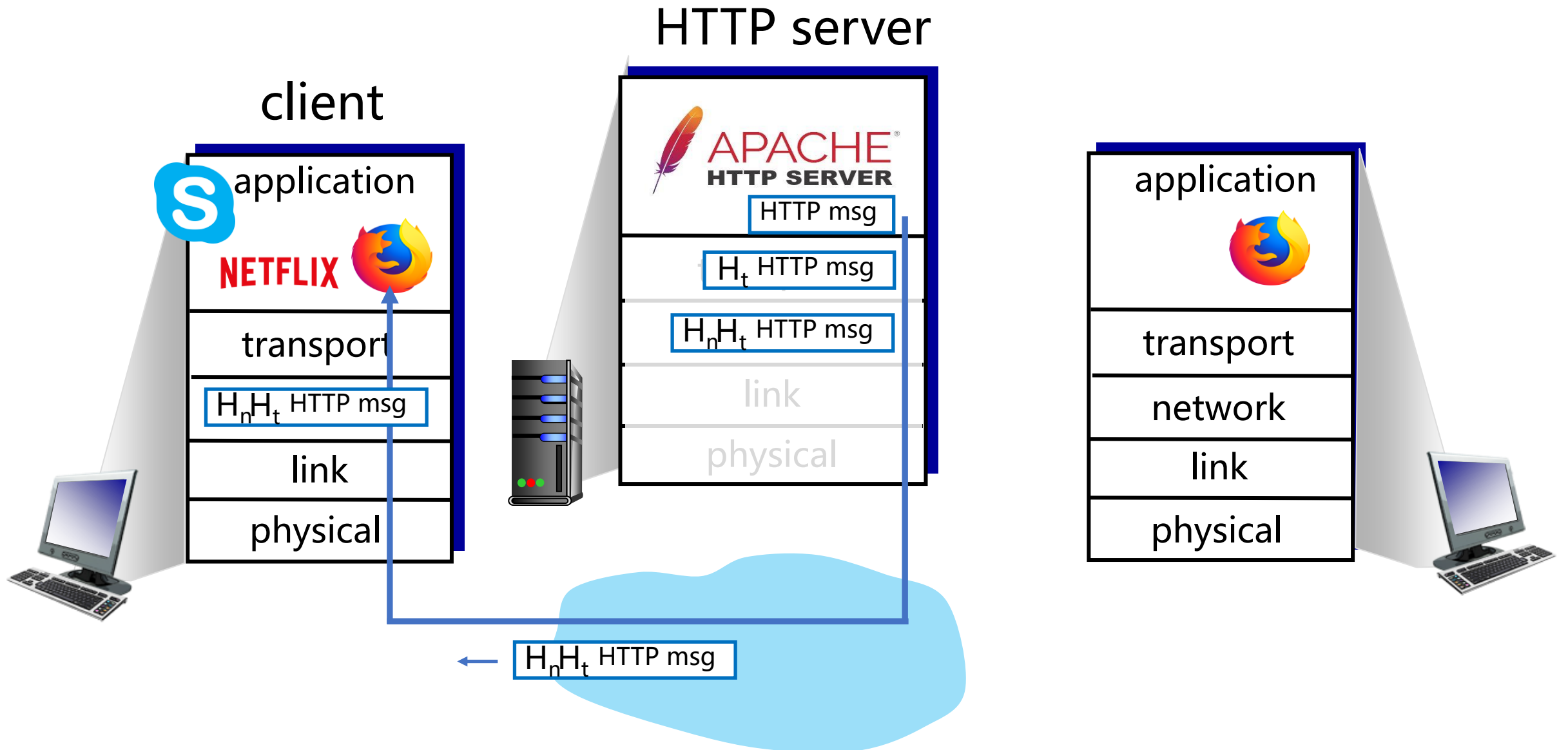
在发送方主机多路复用:

从多个套接字接收来自多个进程的报文, 根据套接字对应的**IP地址**和**端口号**等信息, 对报文段用**头部**加以封装 (该头部信息用于以后的解复用)

在接收方主机多路解复用:

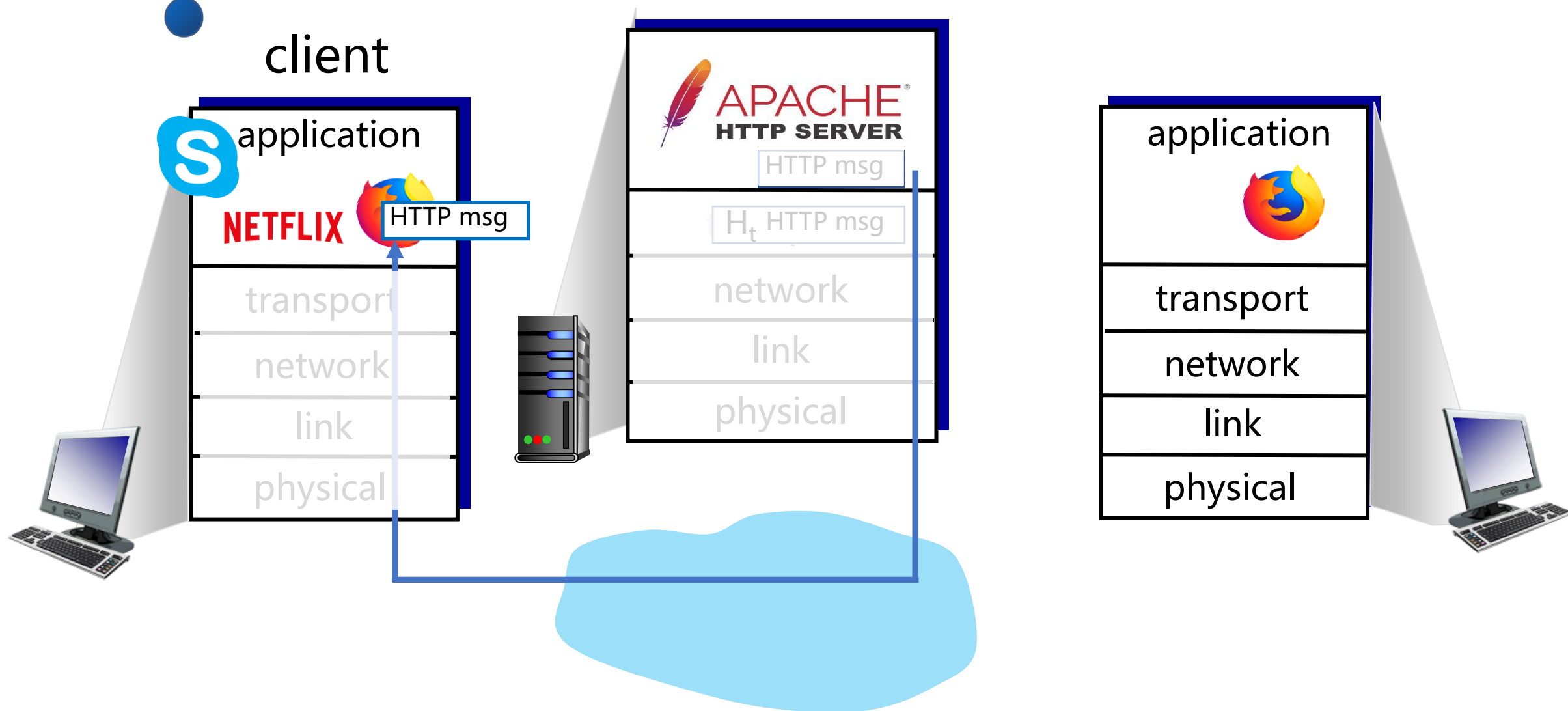
根据报文段的**头部信息**中的**IP地址**和**端口号**将接收到的报文段发给正确的套接字(以及对应的应用进程)





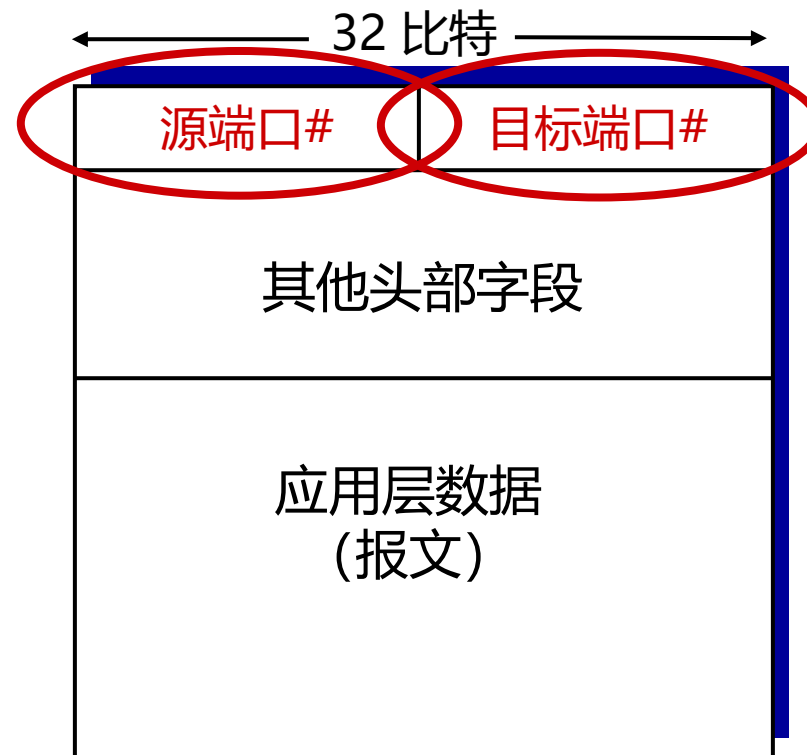


Q: 传输层是如何知道将消息传递给Firefox浏览器进程而不是Netflix进程或Skype进程的?



多路解复用工作原理

- 解复用作用：TCP或者UDP实体采用哪些信息，将报文段的数据部分交给正确的socket，从而交给正确的进程
- 主机收到IP数据报
 - 每个数据报：源IP地址和目标地址
 - 每个数据报：承载一个传输层报文段
 - 每个报文段：源端口号和目标端口号
(特定应用有著名的端口号)
- 主机联合使用**IP地址**和**端口号**将报文段发送给合适的套接字



TCP/UDP 报文段格式

无连接（UDP）多路解复用

回顾:

- 当创建套接字时，必须指定本地端口号（*host-local* port #）：

```
DatagramSocket mySocket1  
= new DatagramSocket(12534);
```

- 当创建UDP报文段采用端口号，必须指定：
 - 目标IP地址
 - 目标端口号

- 当主机接收到UDP报文段时：
 - 检查UDP段中的目标端口号
 - 将UDP段交给对应端口号的套接字



具备相同目标IP地址和目标端口号，即使是不同的源IP地址 或/和源端口号的IP数据报，将会被传到相同的目标UDP套接字上

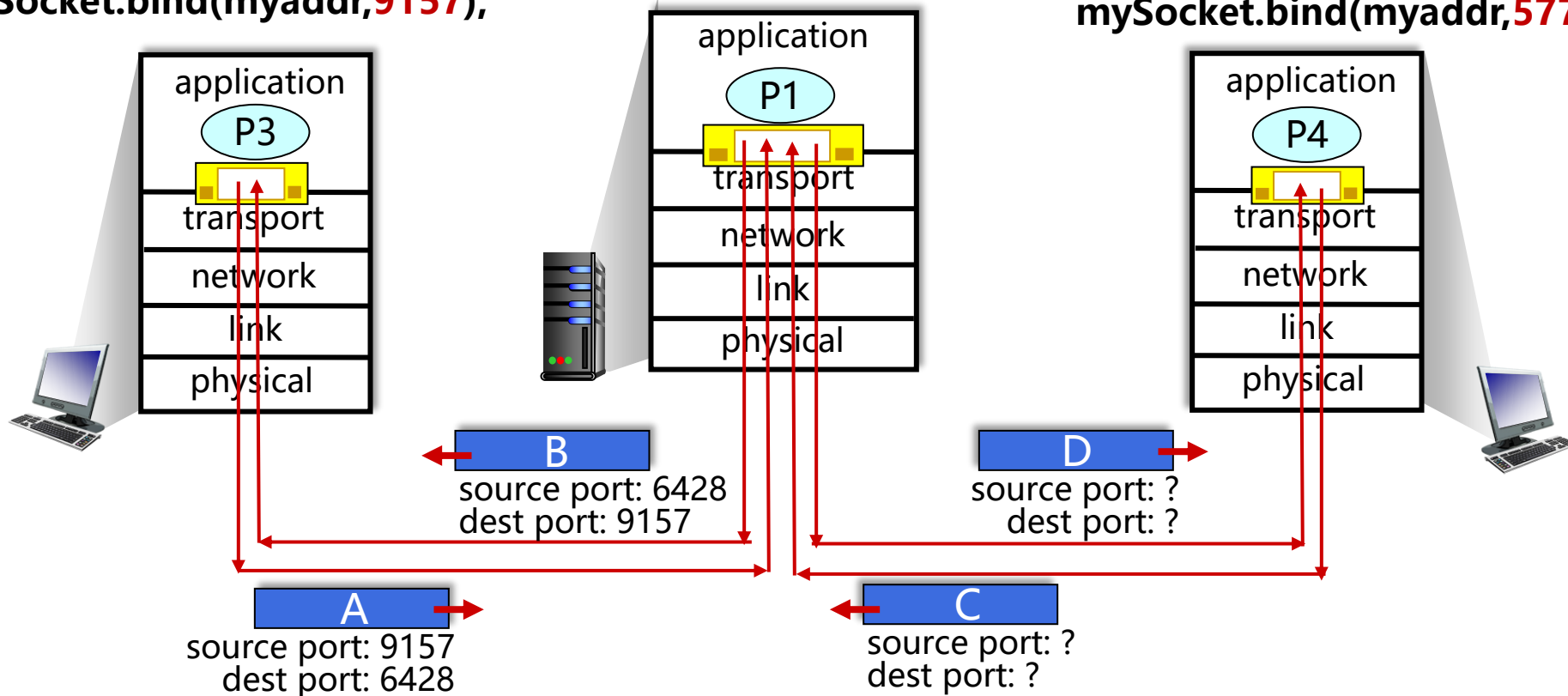
PID	Socket	Des IP	Des Port
12345	77888	202.38.64.1	80
12346	77999	202.38.64.1	11010
.....			

无连接多路复用：例子

```
mySocket =  
    socket(AF_INET,SOCK_DGRAM)  
mySocket.bind(myaddr,6428);
```

```
mySocket =  
    socket(AF_INET,SOCK_DGRAM)  
mySocket.bind(myaddr,9157);
```

```
mySocket =  
    socket(AF_INET,SOCK_DGRAM)  
mySocket.bind(myaddr,5775);
```

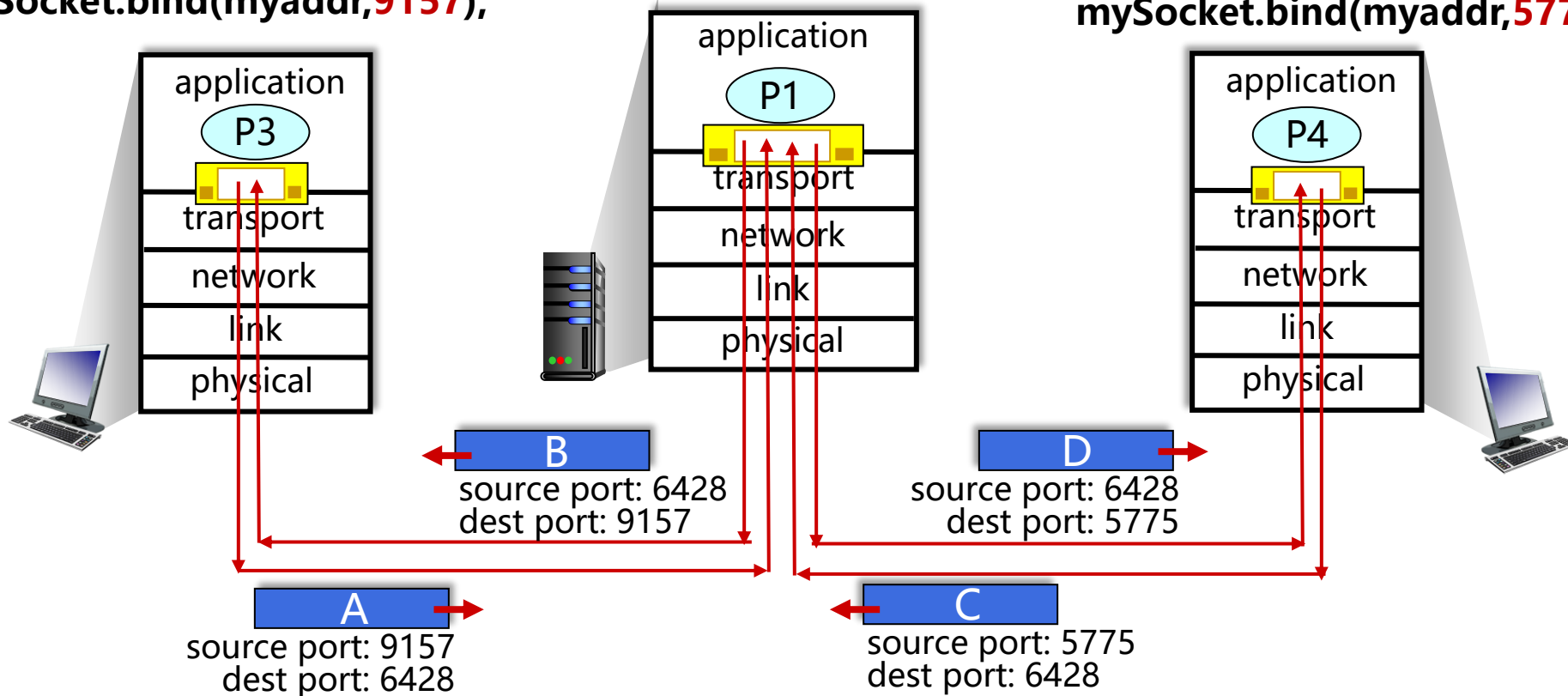


无连接多路复用：例子

```
mySocket =  
    socket(AF_INET,SOCK_DGRAM)  
mySocket.bind(myaddr,6428);
```

```
mySocket =  
    socket(AF_INET,SOCK_DGRAM)  
mySocket.bind(myaddr,9157);
```

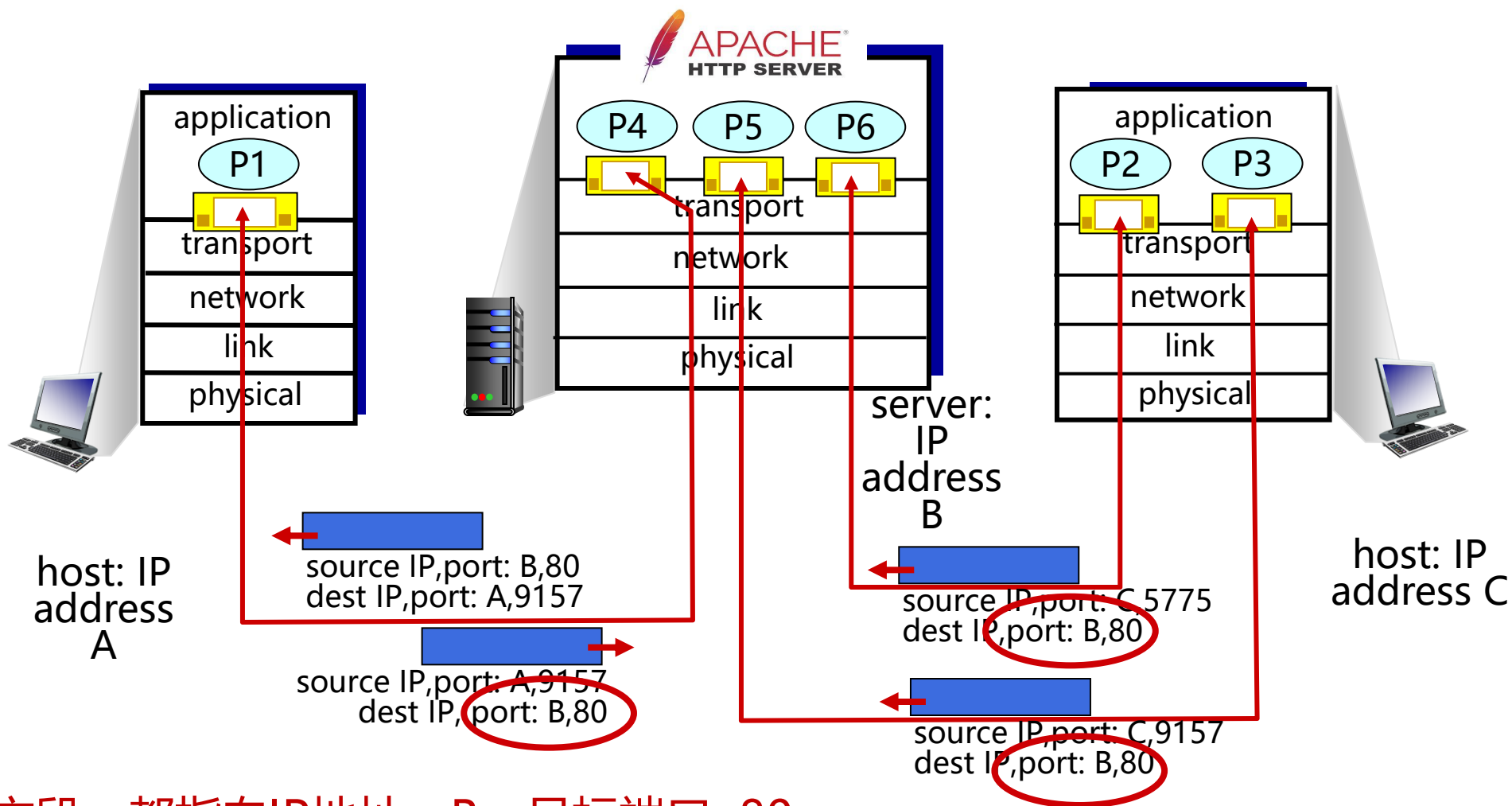
```
mySocket =  
    socket(AF_INET,SOCK_DGRAM)  
mySocket.bind(myaddr,5775);
```



面向连接（TCP）的多路复用

- TCP套接字：四元组本地标识
 - 源IP地址
 - 源端口号
 - 目的IP地址
 - 目的端口号
- 解复用：接收主机用这四个值来将数据报定位到合适的套接字
- 服务器能够在在一个TCP端口上同时支持多个TCP套接字：
 - 每个套接字由其四元组标识（有不同的源IP和源端口）
- Web服务器对每个连接客户端有不同的套接字
 - 非持久对每个请求有不同的套接字

面向连接（TCP）的多路复用：例子



三个报文段，都指向IP地址：B，目标端口：80
被解复用到不同的套接字

小结：多路复用与解复用

- 多路复用、解复用依赖于报文段头部信息
- **UDP:** 仅使用**目的端口号**进行解复用（二元组：**目的IP地址**、**目的端口号**）
- **TCP:** 使用**四元组**进行解复用（**源IP地址**、**源端口号**、**目的IP地址**、**目的端口号**）
- **区别:** 两个具有不同源IP地址和/或源端口号的报文段，
 - UDP报文段：具有相同目的IP地址和端口号，将通过相同的套接字被定向到相同的进程
 - TCP报文段：被定向到两个不同的套接字（除非TCP报文段携带了初始创建连接请求）
- 多路复用、解复用发生在所有层

第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理

3.7 TCP拥塞控制

3.8 传输层功能的演化

UDP: User Datagram Protocol [RFC 768]

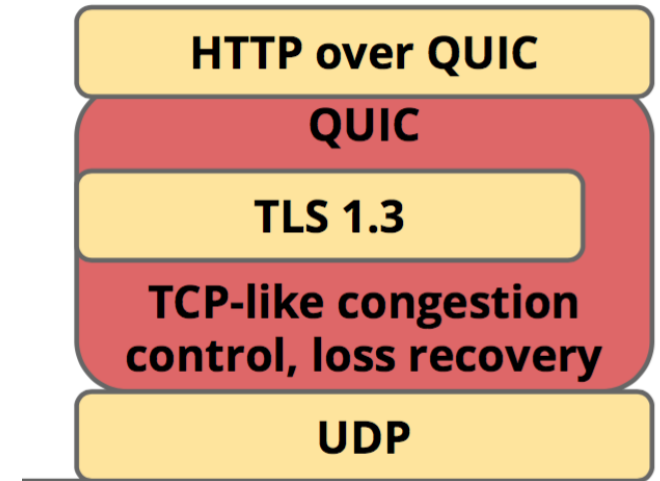
- “no frills,” “bare bones”
Internet 传输协议
- “尽力而为” 的服务，UDP 报文段可能：
 - 丢失
 - 送到应用进程的报文段乱序
- 无连接：
 - 发送端和接收端之间没有握手
 - 每个UDP报文段都被独立地处理

为什么要有UDP?

- 不建立连接（会增加延时）
- 简单：在发送端和接收端没有连接状态
- 报文段的头部很小（开销小）
- 无拥塞控制和流量控制：
 - UDP可以尽可能快的发送报文段
 - 面对拥堵还能正常工作
 - 应用->传输的速率=主机->网络的速率

UDP: 用户数据报协议

- UDP被广泛应用于:
 - 流媒体（丢失不敏感， 速率敏感、 应用可控制 传输速率）
 - DNS
 - SNMP（Simple Network Management Protocol, 简单网络管理协议）
 - HTTP/3
- 如果要在UDP上进行可靠传输（例如 HTTP/3）
 - 在应用层增加可靠性
 - 应用特定的差错恢复



UDP: User Datagram Protocol [RFC 768]

INTERNET STANDARD

RFC 768

J. Postel
ISI
28 August 1980

User Datagram Protocol

Introduction

This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

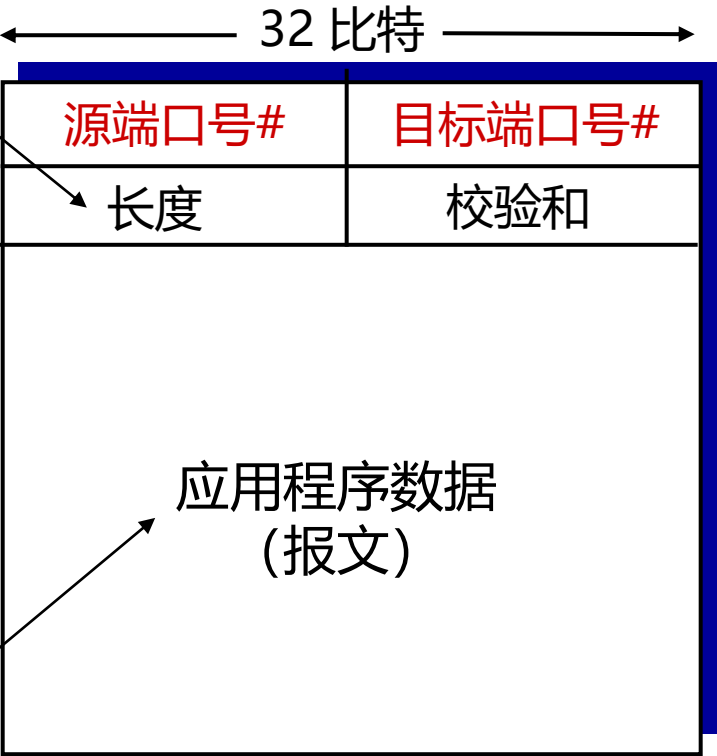
This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

Format

0	7	8	15	16	23	24	31
Source Port				Destination Port			
Length				Checksum			
data octets ...							

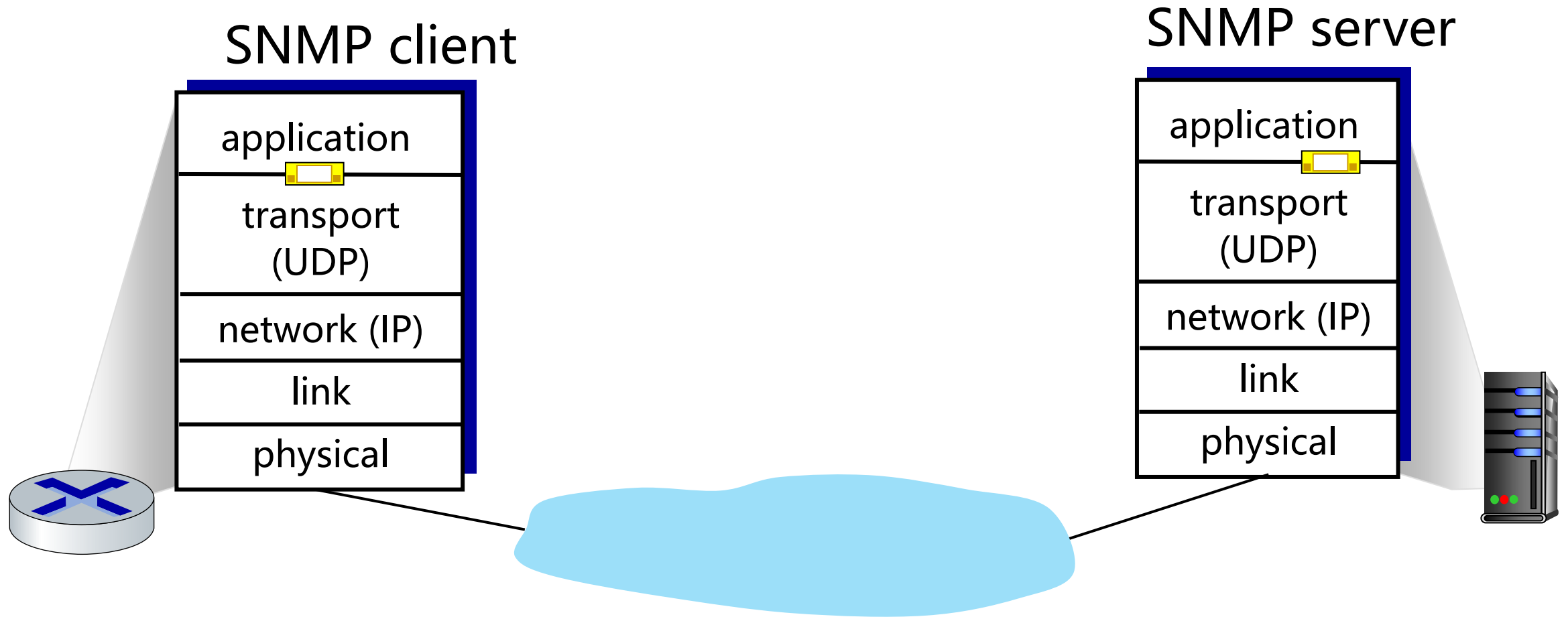
UDP报文段的
字节数，
包括头部

来自传输到
应用层的数
据

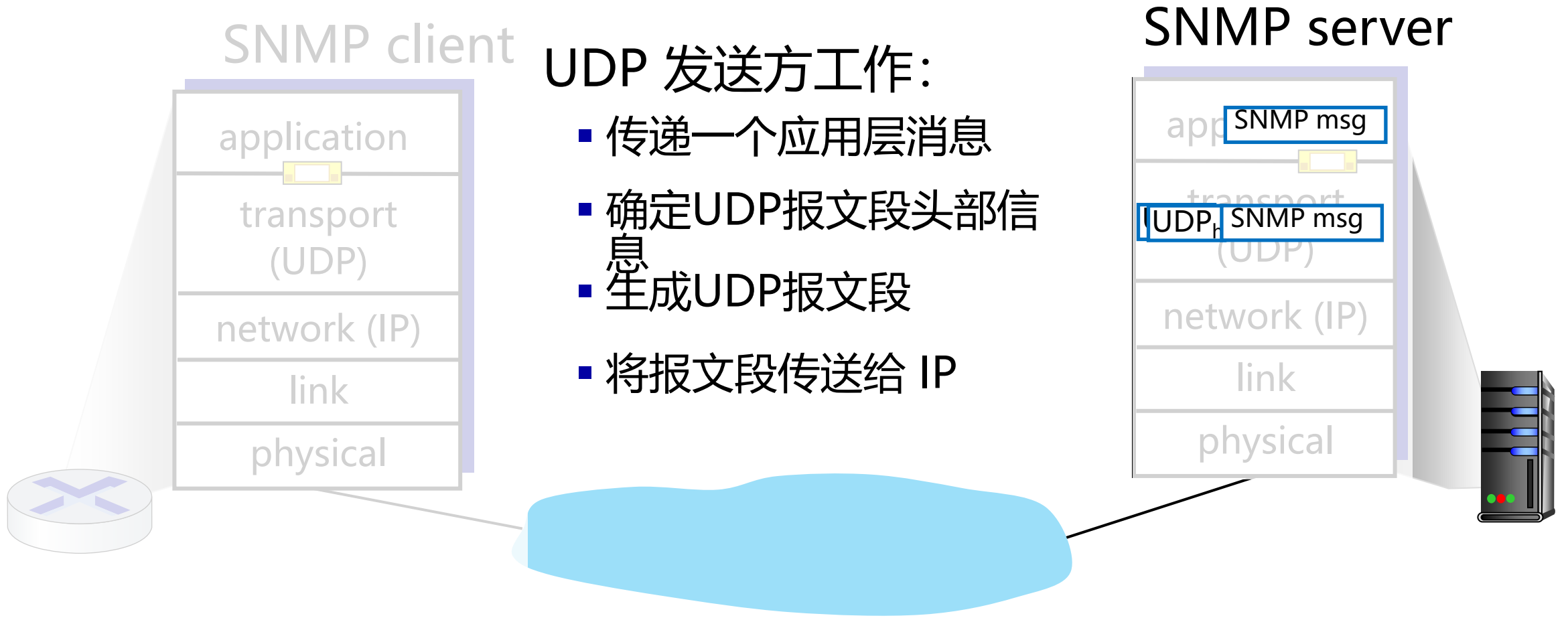


UDP 报文段格式

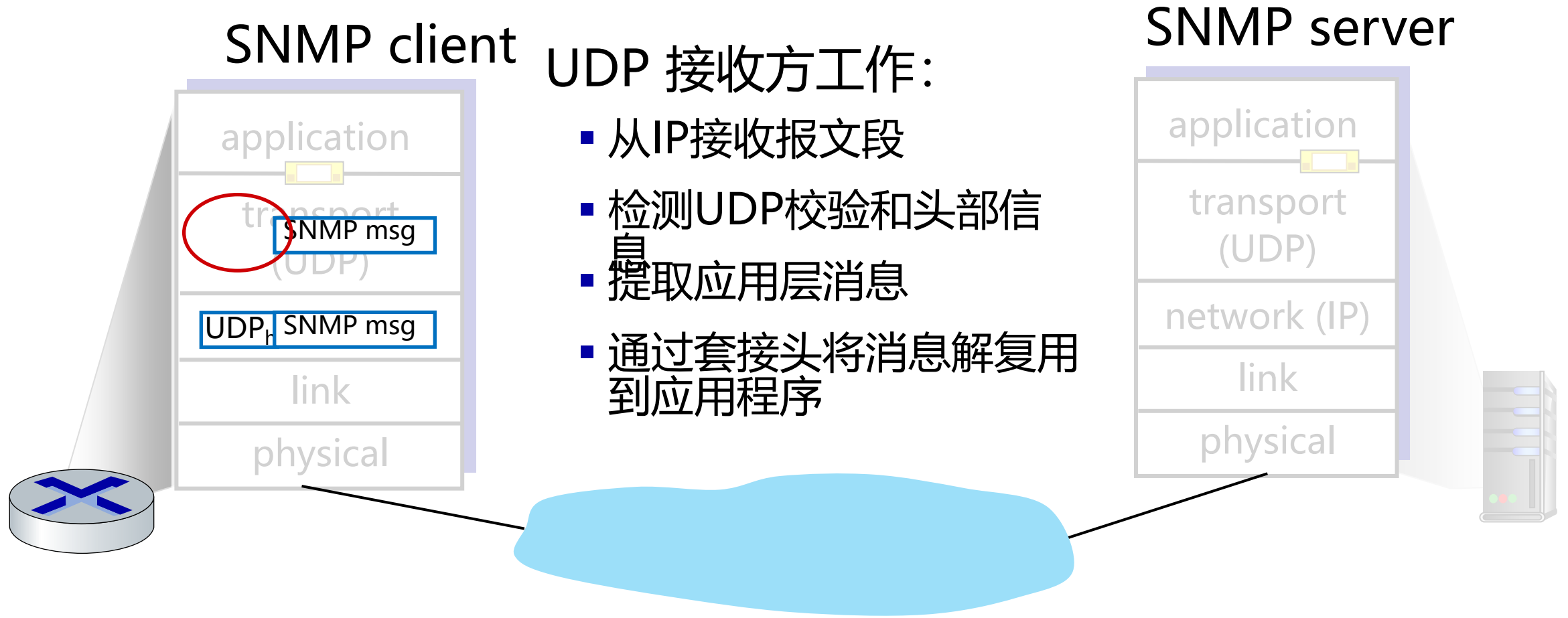
UDP: 传输层工作



UDP: 传输层工作

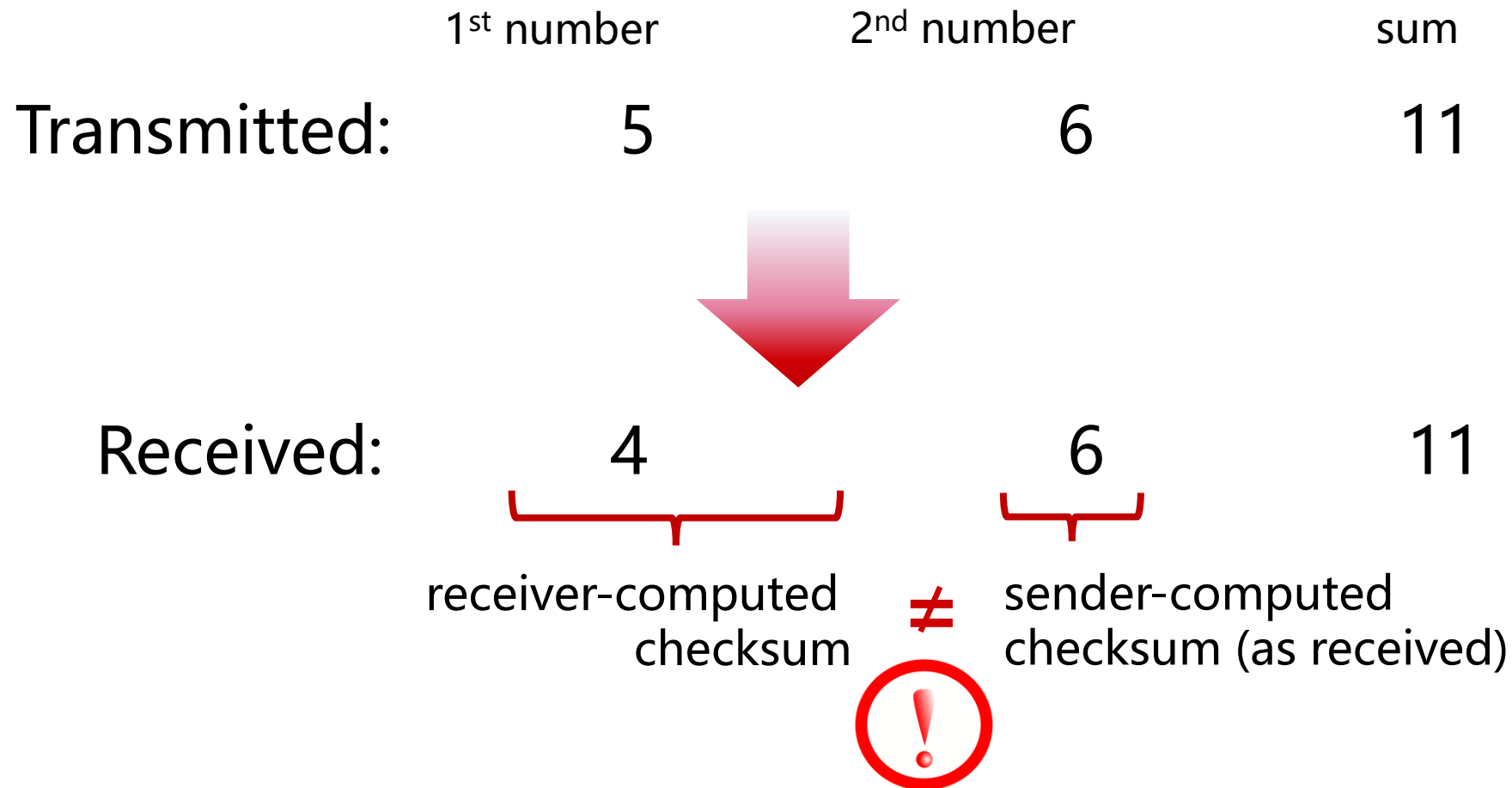


UDP: 传输层工作



UDP 校验和 (Checksum)

目标: 检测在被传输报文段中的差错 (如反码运算)



UDP 校验和 (Checksum)

目标： 检测在被传输报文段中的差错（如反码运算）

发送方：

- 将报文段的内容（UDP头部字段和IP地址）视为16 比特的整数
- **校验和：** 报文段的加法和 (1的补运算)
- 发送方将校验和放在 UDP的校验和字段

接收方：

- 计算接收到的报文段的校验和
- 检查计算出的校验和与校验和字段的内容是否相等：
 - 不相等 - 检测到差错
 - 相等 - **没有检测到差错**，但也许还是有差错
 - 残存错误

Internet 校验和：示例

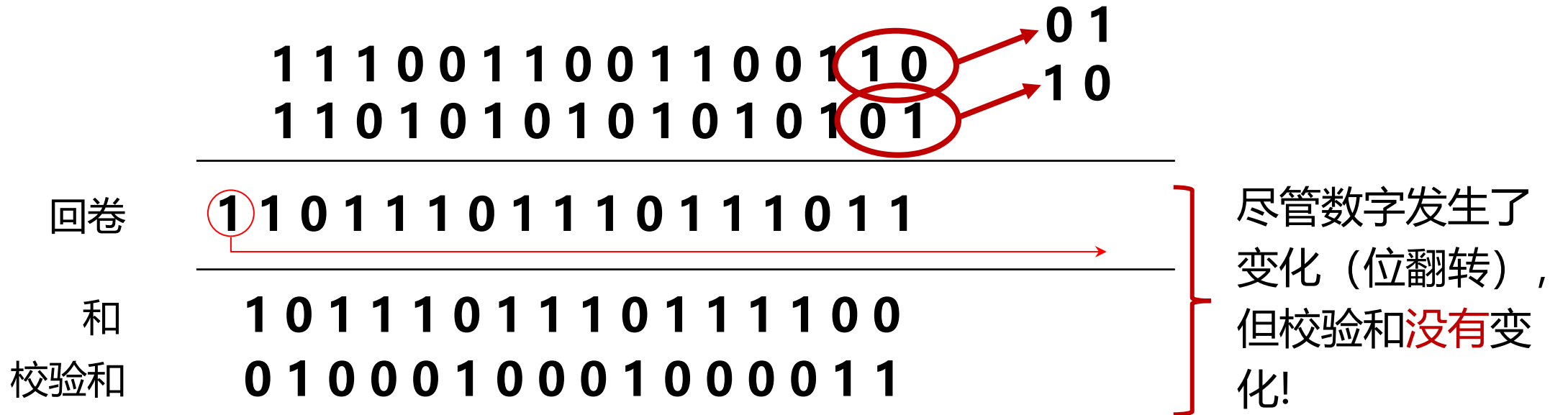
例子：两个16比特的整数相加

	1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
	1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
回卷	1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1
和	1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0
校验和	0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1

- 注意：当数字相加时，在最高位的进位要回卷，再加到结果上
- 目标端：校验范围+校验和=1111111111111111 通过校验
 - 否则不能通过校验。

Internet 校验和：保护弱！

例子：两个16比特的整数相加



小结：无连接传输UDP

- “no frills” 协议：
 - 报文段可能丢失，不按顺序传送
 - 尽力而为服务 (best effort service): “send and hope for the best”
- UDP 优点：
 - 不需要设置或握手（不产生RTT，即返回时延Round-Trip Time）
 - 当网络服务受到威胁时，可以正常工作
 - 有助于提高可靠性（校验和）
- 在应用层的UDP之上构建额外的功能（例如，HTTP/3）

问题&思考

1. 考虑在主机A和主机B之间有一条TCP链接。
 - 假设从主机A传送到主机B的TCP报文段具有源端口号x 和目的端口号y,
 - 对于从主机B传送到主机A的报文段, 源端口号和目的端口号分别是什么?
2. 假定在主机C上的一个进程有一个具有端口号6789的UDP套接字。
 - 假定主机A和主机B都用目的端口号6789向主机C发送一个UDP报文段,
 - 这两台主机的这些报文段在主机C都被描述为相同的套接字吗?
 - 如果是这样的话, 在主机C的该进程将怎样知道源于两台不同主机的这两个报文段?

问题&思考（续）

3. 假定在主机C的端口80上运行着一个Web服务器。
 - 假定这个Web服务器使用持续连接，并且正在接收来自两台不同主机A和B的请求。
 - 被发送的所有请求都通过位于主机C的相同套接字吗？
 - 如果他们通过不同的套接字传递，这两个套接字都具有端口80吗？
4. 描述应用程序开发者可能选择在UDP上运行应用程序而不是在TCP上运行的原因。
5. 在今天的Internet中，为什么语音和图像流量常常是经过TCP而不是经UDP发送？
 - 提示：答案与TCP的拥塞控制机制没有关系。